



Proyecto Final de Ciclo

Ciclo Formativo de Grado Superior de Desarrollo de Aplicaciones Web

IES Sotero Hernández · Curso 2025/2026

Alumno: Joaquín Bizcocho

Acceso al proyecto

Aplicación desplegada: <https://collector-hub-frontend.vercel.app/>

Repositorio GitHub: <https://github.com/JoaquinBizcocho/CollectorHub>

Indice

1. Descripción del sistema.....	2
2. Planteamiento, alcance y requisitos	2
3. Arquitectura Técnica.....	4
4. Documentación técnica.....	5
4.1 Modelo de datos.....	5
4.2 Estructura del backend	6
4.3 Sistema de seguridad	7
4.4 Funcionalidades destacadas.....	9
4.5 Estructura del frontend	10
4.6 Comunicación con el backend	12
4.7 Tabla de endpoints	12
5. Lecciones aprendidas.....	14
6. Planificación preliminar vs real	16
7. Guía de puesta en marcha	17
7.1 Backend	18
7.2 Frontend.....	18
8. Estrategia de Contingencia y Recuperación	18
8.1. Política de Respaldo (Backup)	19
8.2. Procedimiento de Restauración.....	19
9. Ejecución Probada del Plan de Recuperación	19
9.1. Preparación del Entorno Local.....	19
9.2. Inserción de Datos de Prueba	20
9.3. Generación del Backup.....	20
9.4. Destrucción Deliberada de la Base de Datos	22
9.5. Restauración desde el Backup	23
9.6. Verificación de Integridad.....	23
10. Plan de Pruebas y Aseguramiento de Calidad	24
10.1. Pruebas Automáticas.....	24

1. Descripción del sistema

Collector Hub nació de una idea bastante simple: no existe una aplicación que te deje organizar cualquier tipo de colección sin obligarte a adaptarte a su estructura. Si coleccionas cartas necesitas registrar cosas como el estado de conservación o si tiene gradación PSA, si coleccionas libros te interesa el autor o el año de edición. Son necesidades completamente distintas y la mayoría de aplicaciones no contemplan eso.

La solución que he desarrollado permite que cada usuario defina sus propios campos al crear una categoría, de forma que la estructura de datos se adapta a lo que el coleccionista necesita y no al revés. Sobre esa base se construye el resto: añadir artículos con imágenes, separar lo que tienes de lo que quieres conseguir, exportar la colección, importarla desde un fichero o borrar tu cuenta entera si quieres.

También hay un rol de administrador desde el que se pueden crear plantillas oficiales para que otros usuarios las usen como punto de partida, y consultar estadísticas generales de la plataforma.

La aplicación está desplegada en la nube y funciona desde cualquier navegador, también desde el móvil.

2. Planteamiento, alcance y requisitos

Cuando empecé el proyecto tenía claro el concepto general pero no tanto los límites reales de lo que podía llegar a hacer en el tiempo disponible. La planificación inicial era bastante ambiciosa: autenticación con roles, plantillas dinámicas, CRUD completo, exportación de datos, y un sistema de seguimiento de precios históricos con cálculo de inversión total por colección.

La mayoría de esos objetivos se han cumplido, pero el seguimiento de precios históricos no llegó a implementarse. Durante el desarrollo me di cuenta de que requería un rediseño bastante importante del modelo de datos y que el tiempo que le tendría que dedicar comprometería la estabilidad del resto de funcionalidades. Decidí priorizar que todo lo demás funcionara bien antes de añadir algo nuevo a medias.

Lo que sí se ha implementado y funciona de forma completa es lo siguiente:

Registro con verificación por email mediante PIN temporal, inicio de sesión con token JWT y control de acceso por roles. El motor de categorías dinámicas que permite a cada usuario definir sus propios campos. Gestión completa de artículos con soporte de imágenes, separación entre colección y wishlist, y estadísticas por categoría calculadas en el cliente. Exportación e importación de datos en CSV y JSON con detección de conflictos. Panel de administración con plantillas oficiales y métricas globales. Borrado completo de cuenta con eliminación en cascada de todos los datos del usuario, cumpliendo con el derecho de supresión.

Respecto a los requisitos no funcionales, la seguridad se ha trabajado bastante a fondo: contraseñas cifradas con BCrypt, tokens JWT validados en cada petición, extracción del ID de usuario siempre desde el token y nunca desde el cliente, y validación de datos en el backend mediante DTOs anotados. La portabilidad también está cubierta, el proyecto puede ejecutarse en local con Docker o accederse directamente desde la versión desplegada en Render y Vercel.

3. Arquitectura Técnica

El proyecto está dividido en tres partes que funcionan de forma independiente pero conectadas entre sí: el backend, el frontend y la base de datos.

El backend está hecho con Spring Boot y es el que se encarga de toda la lógica: validar usuarios, gestionar los datos y controlar quién puede hacer qué. Para la seguridad se usa JWT, de forma que cada vez que el usuario inicia sesión recibe un token que tiene que adjuntar en todas las peticiones siguientes. Las contraseñas se guardan cifradas con BCrypt, nunca en texto plano. Por dentro el código está organizado en controladores, servicios y repositorios, cada uno con su función, para que no todo esté mezclado en el mismo sitio.

El frontend está desarrollado con React y Vite. Es una SPA, lo que significa que la página no se recarga al navegar entre secciones, todo ocurre en el mismo documento. Las llamadas al backend se hacen desde un archivo centralizado llamado api.js, que añade el token automáticamente en cada petición y redirige al login si la sesión ha caducado. Los datos de sesión se guardan en el LocalStorage del navegador.

La base de datos es MySQL y tiene tres tablas: usuarios, categorías y artículos. Una cosa a destacar es que se usa el tipo JSON de MySQL para guardar la estructura de cada categoría, porque cada colección puede tener campos completamente distintos y no tenía sentido crear una tabla diferente para cada caso. El borrado de cuenta está configurado en cascada, así cuando un usuario se borra, sus categorías y artículos desaparecen también automáticamente.

En producción el backend está desplegado en Render y el frontend en Vercel, y se comunican entre ellos por HTTP igual que lo harían en local.

4. Documentación técnica

4.1 Modelo de datos

La base de datos tiene tres tablas: usuarios, categorías y artículos. Las relaciones son sencillas, cada categoría pertenece a un usuario y cada artículo pertenece a un usuario y a una categoría. El borrado está configurado en cascada a nivel de base de datos, así que cuando se elimina un usuario todo lo suyo desaparece con él sin necesidad de hacerlo manualmente.

Usuario

Además de los campos típicos como alias, correo y contraseña, esta entidad tiene varios campos que solo tienen sentido durante el proceso de registro. Cuando alguien se registra la cuenta no se activa directamente, sino que se guarda con `cuentaActiva` a `false` y se le manda un PIN de seis dígitos al correo. Ese PIN se guarda temporalmente en `codigoVerificacion`, la fecha en que se registró se guarda en `fechaRegistro` para controlar que no lleve más de cinco minutos sin verificar, e `intentosFallidos` lleva la cuenta de las veces que ha metido el PIN mal. Una vez que verifica la cuenta estos campos se limpian y no vuelven a usarse.

Categoria

Tiene nombre, descripción y un campo `esOficial` que distingue las plantillas creadas por el administrador de las colecciones personales de cada usuario. El campo más importante es `esquema`, que se almacena como JSON directamente en MySQL:

```
@JdbcTypeCode(SqlTypes.JSON)
@Column(columnDefinition = "json")
private List<Map<String, String>> esquema;
```

Este campo define qué atributos va a tener cada artículo de esa categoría. Por ejemplo una categoría de cartas puede tener campos como "nombre", "estado" o "gradación PSA", mientras que una de libros tendría "autor", "editorial" o "año". Gracias a esto cada colección tiene su propia estructura sin necesidad de crear tablas nuevas en la base de datos cada vez.

Artículo

Tiene referencias al usuario y a la categoría a la que pertenece, un campo estado que puede ser COLECCION o WISHLIST, y un campo datos también de tipo JSON donde se guardan los valores concretos del artículo siguiendo la estructura que define el esquema de su categoría. Las imágenes se guardan como LONGTEXT en formato Base64, lo que permite almacenarlas directamente en la base de datos sin depender de ningún servidor de archivos externo.

4.2 Estructura del backend

El backend está organizado en tres capas que siguen el patrón Controller → Service → Repository. La idea es que cada capa tenga una responsabilidad clara y no se mezclen cosas que no tienen que ver entre sí.

Los controladores son el punto de entrada de cada petición HTTP. Su trabajo es recibir la petición, extraer el usuario autenticado del token JWT y pasárselo al servicio correspondiente. No toman ninguna decisión de negocio, solo coordinan.

Los servicios son donde está la lógica real. Antes de hacer cualquier cosa comprueban que el recurso exista y que pertenezca al usuario que hace la petición. Si no es así devuelven un 403 o un 404 según el caso. Esto es importante porque evita que un usuario pueda modificar o borrar datos de otro aunque conozca el ID, algo que antes de refactorizar dependía del frontend y era un agujero de seguridad.

Los repositorios son interfaces que extienden JpaRepository y se encargan exclusivamente de hablar con la base de datos. La mayoría de consultas las genera Spring Data automáticamente a partir del nombre del método. Solo en los casos donde eso no era suficiente se ha recurrido a la anotación @Query para escribir la consulta a mano, como en este caso:

```
@Query("SELECT u FROM Usuario u WHERE u.cuentaActiva = false AND u.fechaRegistro < ?1")
List<Usuario> buscarCuentasExpiradas(LocalDateTime fechaLimite);
```

Esta consulta la usa un scheduler que se ejecuta cada minuto para limpiar los usuarios que llevan más de cinco minutos sin verificar su cuenta, evitando que se acumulen registros a medias en la base de datos.

En cuanto a los servicios en detalle, `CategoriaService` gestiona el ciclo de vida completo de las colecciones y distingue entre categorías personales y plantillas oficiales del administrador. `ArticuloService` es el más extenso porque además del CRUD incluye la lógica de exportación e importación. Al exportar construye el fichero JSON o CSV en memoria recorriendo los artículos de la categoría. Al importar primero detecta si hay conflictos entre los IDs del fichero y los artículos ya existentes, y si los hay sin que el usuario haya confirmado la sobreescritura devuelve una respuesta intermedia para que el frontend pueda preguntarle antes de continuar.

4.3 Sistema de seguridad

La seguridad es una de las partes en las que más tiempo invertí durante el desarrollo, sobre todo porque al principio el control de acceso dependía casi todo del frontend, lo cual es un problema real. Si alguien sabía la URL podía hacer peticiones directamente al backend sin pasar por la interfaz. Eso lo corregí integrando Spring Security con JWT.

El flujo es el siguiente: cuando el usuario inicia sesión correctamente el servidor genera un token firmado con una clave secreta que solo conoce el servidor y que contiene el id, el alias y el rol del usuario. Ese token caduca en 24 horas y el frontend lo guarda en el `LocalStorage` para adjuntarlo en todas las peticiones siguientes.

La generación del token se hace así:

```
return Jwts.builder()
    .setSubject(alias)
    .claim(s: "id", usuarioId)
    .claim(s: "rol", rol)
    .setIssuedAt(new Date())
    .setExpiration(new Date(System.currentTimeMillis() + 86400000))
    .signWith(getKey(), SignatureAlgorithm.HS256)
    .compact();
```

Cada petición que llega al servidor pasa por `JwtFilter` antes de llegar al controlador. Este filtro lee el header `Authorization`, extrae el token, valida su firma y si todo es correcto mete el usuario autenticado en el `SecurityContext` de Spring para que los controladores puedan usarlo directamente. Si el token no es válido o no existe simplemente no se autentica y Spring Security bloquea la petición.

Una decisión importante fue crear la clase `AuthenticatedUser` para guardar dentro del `SecurityContext` no solo el alias sino también el id y el rol del usuario. Así los controladores pueden extraer el id directamente del token en lugar de recibirlo del cliente, lo que evita que un usuario pueda manipular las peticiones para acceder a datos de otro:

```
AuthenticatedUser principal = new AuthenticatedUser(usuarioId, alias, rol);
```

Las reglas de acceso están definidas en `SecurityConfig`. Solo los endpoints de registro y login son públicos, las rutas de administración requieren explícitamente el rol `ADMIN`, y el resto requieren cualquier usuario autenticado:

```
.requestMatchers(🔒"/api/auth/**").permitAll()  
.requestMatchers(🔒"/api/admin/**").hasRole("ADMIN")  
.anyRequest().authenticated()
```

El servidor está configurado como `stateless`, lo que significa que no guarda ninguna sesión en memoria. Cada petición se autentica de forma independiente a partir del token, lo cual es lo correcto para una API REST. El CSRF está desactivado porque con JWT no tiene sentido mantenerlo, y la política CORS está configurada para aceptar peticiones desde los orígenes permitidos tanto en local como en producción.

Las contraseñas se cifran con `BCrypt` antes de guardarse en la base de datos y nunca viajan en texto plano en ningún momento del flujo.

4.4 Funcionalidades destacadas

Flujo de registro con verificación por PIN

El registro tiene más lógica de lo que parece a primera vista. Cuando alguien se registra la cuenta se guarda pero con `cuentaActiva` a `false`, es decir no puede entrar todavía. Se genera un PIN de seis dígitos y se manda al correo del usuario. Usé `SecureRandom` porque `Random` no es adecuado para generar códigos de verificación:

```
String pin = String.format("%06d", new SecureRandom().nextInt( bound: 1000000));
```

El usuario tiene cinco minutos y tres intentos. Si se equivoca tres veces o se le pasa el tiempo la cuenta se borra y tiene que volver a registrarse. Para limpiar esas cuentas caducadas hay un scheduler que mira cada minuto si hay usuarios sin verificar con más de cinco minutos de antigüedad y los elimina:

```
@Scheduled(fixedRate = 60000) // Joaquín Bizcocho Cano *
public void limpiarCuentasNoVerificadas() {
    LocalDateTime hace5Minutos = LocalDateTime.now().minusMinutes(5);
    List<Usuario> expirados = usuarioRepository
        .buscarCuentasExpiradas(hace5Minutos);
    if (!expirados.isEmpty()) {
        usuarioRepository.deleteAll(expirados);
        System.out.println("Limpieza: eliminados " + expirados.size() + " usuario(s) sin verificar.");
    }
}
```

Hay un caso que me dio bastante guerra al principio: si el envío del correo falla la cuenta se quedaba guardada a medias en la base de datos y la persona no podía volver a intentarlo porque el alias o el correo ya estaban pillados. Lo solucione borrando el usuario inmediatamente si el envío falla.

Imágenes en Base64

Cada artículo puede tener hasta dos imágenes. El frontend las convierte a Base64 y el backend las guarda como LONGTEXT en MySQL. No hay servidor de archivos ni nada externo, las imágenes van dentro del mismo registro del artículo.

Lo bueno es que es muy simple y el backup incluye las imágenes sin hacer nada especial. Lo malo es que Base64 ocupa bastante más que el archivo original y con muchos artículos y muchas fotos la base de datos crece rápido. Para este proyecto funciona bien pero es algo que cambiaría en una versión futura.

Exportación e importación con detección de conflictos

La exportación genera un JSON o CSV con los artículos de una categoría. En el CSV las claves del campo datos se usan como cabeceras de columna, cogiendo las del primer artículo como referencia.

La importación tiene una lógica para evitar machacar datos sin querer. Antes de crear nada compara los IDs del fichero con los que ya hay en la categoría, y si hay coincidencias para y avisa al frontend en vez de sobrescribir directamente:

```
if (!conflictos.isEmpty() && !sobrescribir) {  
    Map<String, Object> respuesta = new LinkedHashMap<>();  
    respuesta.put("conflictos", conflictos.size());  
    respuesta.put("requiereConfirmacion", true);  
    return respuesta;  
}
```

El frontend muestra un aviso con cuántos conflictos hay y pregunta si quiere sobrescribir. Si dice que sí se hace la misma petición con sobrescribir a true y entonces sí actualiza los existentes y crea los nuevos. Si el fichero no tiene el formato correcto el backend lo rechaza con un 400.

4.5 Estructura del frontend

Para la navegación entre pantallas no usé React Router, simplemente tengo un estado en App.jsx que guarda en qué vista está el usuario en cada momento y renderiza el componente correspondiente. Al principio no tenía claro si esto era la mejor solución pero para el tamaño del proyecto funciona bien y no añade dependencias innecesarias.

```
const [vistaActual, setVistaActual] = useState(() => {  
    const usuarioGuardado = localStorage.getItem('usuarioId');  
    return usuarioGuardado ? 'dashboard' : 'login';  
});
```

Cuando la app arranca mira el LocalStorage para ver si el usuario ya tenía sesión y si es así lo manda directamente al dashboard. El rol lo saco del token JWT en vez del LocalStorage porque si lo guardas en LocalStorage cualquiera puede cambiarlo desde las devtools del navegador:

```
const payload = JSON.parse(atob(token.split('.')[1]));  
return payload.rol || 'user';
```

Si alguien intenta entrar al panel de admin manipulando el LocalStorage el backend le devuelve un 403 igualmente porque el token no tiene ese rol.

Los tres componentes principales son `CategoriasDashboard`, `ArticulosView` y `AdminDashboard`. El de categorías tiene un modal con dos pasos donde el usuario elige si quiere crear su colección desde cero o partir de una plantilla oficial. El de admin solo se monta si el token tiene rol de admin, si no ni aparece.

`ArticulosView` fue el que más tiempo me llevó. Tiene bastante dentro: las pestañas de colección y wishlist, la ordenación por campos, el carrusel de imágenes, el lightbox, el panel de estadísticas lateral... Lo del panel de estadísticas lo hice para que calculara todo en el cliente con los artículos ya cargados, así no tenía que hacer más peticiones al servidor cada vez que alguien miraba un número. La subida de imágenes la dejé con botón, drag and drop y también pegando con Ctrl+V, lo del portapapeles lo añadí casi por probar y quedó bien.

4.6 Comunicación con el backend

Todas las llamadas al servidor van a través de api.js. Lo separé por módulos: categoriasApi, articulosApi, adminApi y usuarioApi, para no tener fetch sueltos por todos los componentes como tenía al principio.

La URL del backend la coge de una variable de entorno de Vite, así en local apunta a localhost y en producción a Render sin tocar el código:

```
const API_BASE = import.meta.env.VITE_API_URL || 'http://localhost:8080';
```

Hay una función fetchConAuth que es la que hace todos los fetch. Añade el token automáticamente y si el servidor devuelve un 401 limpia la sesión y manda al usuario al login. Antes de tener esto cuando el token expiraba el usuario se quedaba con errores raros sin entender qué pasaba:

```
const fetchConAuth = async (url, opciones = {}) => {  
  const response = await fetch(url, opciones);  
  if (response.status === 401) {  
    cerrarSesionAutomatico();  
  }  
  return response;  
};
```

La redirección al login se hace con un callback que App.jsx registra al arrancar. api.js no sabe nada de React ni del estado de la app, solo llama al callback cuando toca.

4.7 Tabla de endpoints

A continuación se recogen todos los endpoints disponibles en la API, indicando el método HTTP, la ruta, si requiere autenticación y lo que hace cada uno.

Método	Endpoint	Auth	Descripción
POST	/api/auth/login	No	Inicia sesión y devuelve el token JWT
POST	/api/auth/register	No	Registra un nuevo usuario y manda el PIN al correo
POST	/api/auth/verify-pin	No	Verifica el PIN e activa la cuenta
GET	/api/categorias/usuario	Sí	Devuelve las categorías del usuario autenticado
GET	/api/categorias/oficiales	Sí	Devuelve las plantillas oficiales del admin
POST	/api/categorias	Sí	Crea una categoría nueva
PUT	/api/categorias/{id}	Sí	Actualiza una categoría existente
DELETE	/api/categorias/{id}	Sí	Elimina una categoría y sus artículos
GET	/api/articulos/categoria/{id}	Sí	Devuelve los artículos de una categoría
POST	/api/articulos	Sí	Crea un artículo nuevo
PUT	/api/articulos/{id}	Sí	Actualiza un artículo existente
DELETE	/api/articulos/{id}	Sí	Elimina un artículo
GET	/api/articulos/categoria/{id}/exportar/json	Sí	Exporta la colección en formato JSON
GET	/api/articulos/categoria/{id}/exportar/csv	Sí	Exporta la colección en formato CSV

POST	/api/articulos/categoria/{id}/importar/json	Sí	Importa artículos desde un fichero JSON
POST	/api/articulos/categoria/{id}/importar/csv	Sí	Importa artículos desde un fichero CSV
GET	/api/admin/estadisticas	Solo admin	Devuelve métricas globales del sistema
DELETE	/api/usuario/cuenta	Sí	Elimina la cuenta y todos sus datos

5. Lecciones aprendidas

Lo más difícil del proyecto no fue el código en sí sino tomar decisiones. Tenía claro lo que quería hacer pero a la hora de implementarlo había muchas formas de hacerlo y no siempre sabía cuál era la mejor. Por ejemplo con el tema de la seguridad estuve un tiempo con todo el control en el frontend hasta que me di cuenta de que eso no servía de nada, cualquiera podía saltárselo. Eso me obligó a rehacer bastante código que ya tenía funcionando, algo que habría evitado si hubiera pensado más en la arquitectura desde el principio.

Qué repetiría

El uso de Railway para la base de datos fue una buena decisión, poder trabajar desde cualquier sitio sin depender de tener Docker levantado en un ordenador concreto me facilitó mucho las cosas. También el hecho de tener la bitácora, porque cuando surgía un error podía mirar atrás y ver si algo similar ya me había pasado antes.

Qué no volvería a hacer

Empezar a desarrollar sin pensar suficiente en el usuario final. Me centré mucho en que las cosas funcionaran técnicamente y menos en cómo las iba a usar alguien de verdad. Cuando le enseñé la app a gente me pedían cosas que no había contemplado, como

poder destacar artículos o personalizar los colores. Si lo hubiera pensado antes habría tomado algunas decisiones de diseño distintas.

También empezaría antes con la capa de servicios y los DTOs. Los añadí casi al final y supuso refactorizar bastante código. Si lo hubiera hecho desde el principio me habría ahorrado ese trabajo.

Tres mejoras para una versión 2

La primera sería un panel de administración de usuarios real, donde desde la propia aplicación se pudieran gestionar cuentas, cambiar roles o ver quién está registrado. Ahora mismo eso solo se puede hacer directamente desde la base de datos, lo cual no es práctico.

La segunda sería dejar que el usuario personalice el aspecto de la aplicación. El verde que elegí me parece que encaja bien con la idea de una app para organizarte y estar tranquilo, pero el feedback que recibí es que a la gente le gustaría poder elegir colores o al menos tener alguna alternativa.

La tercera sería un sistema de planes. Dependiendo del plan el usuario tendría un límite de categorías o artículos, lo que abriría la posibilidad de monetizar la plataforma de forma sencilla sin complicar demasiado la arquitectura.

6. Planificación preliminar vs real

La planificación inicial dividía el proyecto en fases semanales bastante ordenadas: primero la base de datos y Docker, luego el backend, después la seguridad y por último el cierre. Sobre el papel tenía sentido pero en cuanto empecé a desarrollar me di cuenta de que trabajar así de compartimentado no era tan fácil.

El backend y el frontend se fueron desarrollando casi en paralelo desde el principio. Necesitaba ver resultados reales en la interfaz para validar que el backend funcionaba, y al revés, según iba avanzando el frontend me daba cuenta de cosas que tenía que cambiar en el backend. No tenía sentido terminar uno antes de tocar el otro así que fui alternando según lo que necesitaba en cada momento.

Hubo decisiones que surgieron sobre la marcha y que no estaban en ningún sitio de la planificación original. Algunas porque las circunstancias lo pedían, como la migración a Railway que vino por no poder trabajar bien con Docker en el ordenador del trabajo. Otras porque al ir desarrollando me di cuenta de que lo que tenía no era suficientemente bueno, como la refactorización de seguridad donde introduje los DTOs y la capa de servicios casi al final. Y una, el seguimiento de precios históricos, que simplemente no llegué a hacer porque el tiempo no dio para todo.

Las desviaciones más importantes respecto a lo planificado fueron estas:

Tarea	Estado
Migración de Docker a Railway	No estaba planificado, se decidió sobre la marcha por restricciones de red
Verificación por email con PIN	No estaba planificado, añadió bastante trabajo al bloque de seguridad
DTOs y capa de servicios	No estaba planificado, se refactorizó casi al final
Exportación e importación CSV/JSON	Estaba planificado pero se implementó más tarde de lo previsto
Seguimiento de precios históricos	Estaba planificado pero no llegó a implementarse

En general el proyecto se entregó en el plazo previsto aunque el camino fue bastante diferente al cronograma inicial. Algunas cosas que no estaban planificadas acabaron siendo mejoras importantes, y otras que sí estaban planificadas tuvieron que quedarse fuera por falta de tiempo.

7. Guía de puesta en marcha

7.1 Backend

Lo primero es configurar las variables de entorno porque sin eso el proyecto no arranca. En el repositorio hay un archivo `application.properties.example` que muestra exactamente qué hace falta: la conexión a la base de datos, el secreto JWT y las claves de EmailJS. El secreto JWT tiene que ser una cadena larga, si lo pones corto Spring lo rechaza al arrancar.

Si se ejecuta en local desde IntelliJ las variables se configuran en Edit Configurations, dentro de las opciones de ejecución del proyecto. Una cosa importante: la base de datos está en Railway, no hace falta montar nada en local para eso. Solo hay que asegurarse de que los parámetros de conexión del `.properties` apuntan a la instancia de Railway correctamente.

Con eso listo el proyecto arranca desde IntelliJ y queda escuchando en el puerto 8080.

7.2 Frontend

Entrar en la carpeta del frontend e instalar dependencias:

`npm install`

Crear un archivo `.env` en la raíz con la URL del backend. Si se va a probar en local:

`VITE_API_URL=http://localhost:8080`

Y arrancar:

`npm run dev`

La app queda en `http://localhost:5173`. Si se quiere probar contra el backend de producción en Render solo hay que cambiar esa variable por la URL de Render, sin tocar nada más del código.

8. Estrategia de Contingencia y Recuperación

8.1. Política de Respaldo (Backup)

Como las imágenes están guardadas en Base64 dentro de la propia base de datos, hacer un backup de MySQL es suficiente para tenerlo todo: los datos, los esquemas dinámicos y las fotos. No hay que preocuparse de hacer copias de ningún servidor de archivos externo porque no hay ninguno.

El backup se genera con mysqldump sin necesidad de parar el servicio:

```
docker exec [id_contenedor_mysql] mysqldump -u root -p[password] collectorhub > backup_diario.sql
```

8.2. Procedimiento de Restauración

Si se pierde la base de datos el proceso de recuperación es levantar un contenedor limpio con docker-compose y volcar el SQL que se generó antes:

```
docker exec -i [id_contenedor_mysql] mysql -u root -p[password] collectorhub < backup_diario.sql
```

Al restaurar el volcado completo los esquemas JSON y las imágenes vuelven exactamente igual que estaban, no hay nada que reconstruir por separado.

9. Ejecución Probada del Plan de Recuperación

9.1. Preparación del Entorno Local

Lo primero fue levantar el contenedor MySQL en local. Aquí tuve algún problema al principio porque Docker Desktop no estaba arrancado y después el contenedor daba conflicto porque ya existía uno con el mismo nombre de una sesión anterior. Lo resolví eliminando el contenedor viejo y volviendo a lanzarlo:

```
docker rm -f ch_database
docker-compose up -d
docker ps
```

Una vez resuelto, el contenedor ch_database arrancó sin problemas y quedó escuchando en el puerto 3307 de mi máquina.

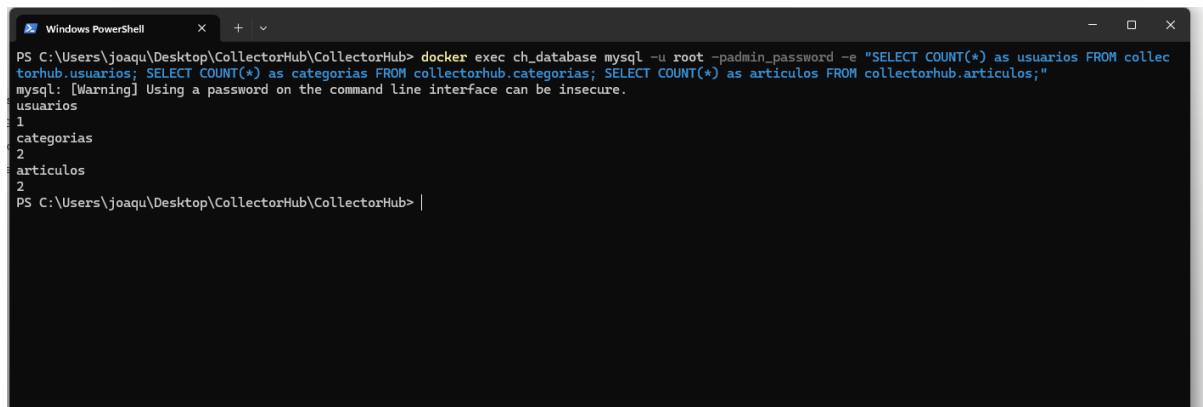
```
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
8cc64e2c4700   mysql:8.0      "docker-entrypoint.s..." 24 seconds ago Up 23 seconds 0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp ch_database
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub>
```

9.2. Inserción de Datos de Prueba

Con la base de datos vacía y las tablas ya creadas por el `init.sql`, inserté datos de prueba para tener algo real que respaldar. Como el backend estaba apuntando a Railway, opté por insertar los datos directamente con comandos SQL desde la terminal, usando un archivo `.sql` intermedio para evitar problemas con las comillas del JSON en PowerShell.

Los datos que metí fueron:

- 1 usuario llamado joaquin con rol user y cuenta activa
- 2 categorías: Cartas Pokemon y Libros Raros, cada una con su esquema JSON de atributos
- 2 artículos: uno con datos de una carta y otro con datos de un libro, ambos en formato JSON



```
Windows PowerShell
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub> docker exec ch_database mysql -u root -padmin_password -e "SELECT COUNT(*) as usuarios FROM collectorhub.usuarios; SELECT COUNT(*) as categorias FROM collectorhub.categorias; SELECT COUNT(*) as articulos FROM collectorhub.articulos;"
mysql: [Warning] Using a password on the command line interface can be insecure.
usuarios
1
categorias
2
articulos
2
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub>
```

9.3. Generación del Backup

Con los datos dentro, lancé el `mysqldump` para generar el volcado completo de la base de datos:

Comprobé que el archivo se había creado correctamente:

```
dir backup_diario.sql
```

Al abrirlo en VS Code pude comprobar que contiene las sentencias `CREATE TABLE` de las tres tablas y los `INSERT INTO` con todos los registros, incluyendo los campos JSON con los esquemas de las categorías y los datos de los artículos.

```
Windows PowerShell
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub> dir backup_diario.sql

Directorio: C:\Users\joaqu\Desktop\CollectorHub\CollectorHub

Mode                LastWriteTime         Length Name
----                -
-a-----         10/05/2026      18:36           9520 backup_diario.sql

PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub> |
```

```
backup_diario.sql X
C:\Users\joaqu\Desktop\CollectorHub\CollectorHub> backup_diario.sql
1  -- MySQL dump 10.13  Distrib 8.0.45, for Linux (x86_64)
2  --
3  -- Host: localhost    Database: collectorhub
4  --
5  -- Server version 8.0.45
6
7  /*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
8  /*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
9  /*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
10 /*!50503 SET NAMES utf8mb4 */;
11 /*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
12 /*!40103 SET TIME_ZONE='+00:00' */;
13 /*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
14 /*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
15 /*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
16 /*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;
17
18 --
19 -- Table structure for table `articulos`
20 --
21
22 DROP TABLE IF EXISTS `articulos`;
23 /*!40101 SET @saved_cs_client = @@character_set_client */;
24 /*!50503 SET character_set_client = utf8mb4 */;
25 CREATE TABLE `articulos` (
26   `id` int NOT NULL AUTO_INCREMENT,
27   `categoria_id` int NOT NULL,
28   `usuario_id` bigint NOT NULL,
29   `datos` json DEFAULT NULL,
30   `imagen1` longtext,
31   `imagen2` longtext,
32   PRIMARY KEY (`id`),
33   KEY `categoria_id` (`categoria_id`),
34   KEY `usuario_id` (`usuario_id`),
35   CONSTRAINT `articulos_ibfk_1` FOREIGN KEY (`categoria_id`) REFERENCES `categorias` (`id`) ON DELETE CASCADE,
36   CONSTRAINT `articulos_ibfk_2` FOREIGN KEY (`usuario_id`) REFERENCES `usuarios` (`id`) ON DELETE CASCADE
37 ) ENGINE=InnoDB AUTO_INCREMENT=3 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
38 /*!40101 SET character_set_client = @saved_cs_client */;
39
40 --
41 -- Dumping data for table `articulos`
42 --
43
44 LOCK TABLES `articulos` WRITE;
45 /*!40000 ALTER TABLE `articulos` DISABLE KEYS */;
46 INSERT INTO `articulos` VALUES (1,1,1, '{"tipo": "Fuego", "numero": "001"}', NULL, NULL), (2,2,1, '{"anio": "1605", "autor": "Cervantes"}', NULL,
47   );
48 UNLOCK TABLES;
```

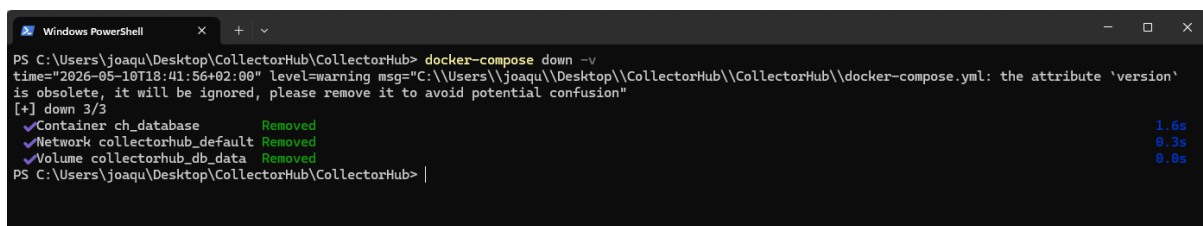
9.4. Destrucción Deliberada de la Base de Datos

Esta es la parte más importante del simulacro. Para demostrar que la recuperación funciona de verdad, no me bastaba con borrar solo los registros. Usé el flag -v de docker-compose down para eliminar también el volumen persistente donde MySQL guarda los datos, que es lo que pasaría en un fallo real de disco o un borrado accidental del entorno:

```
docker-compose down -v
```

La terminal confirmó que se eliminaron el contenedor, la red y el volumen:

- Container ch_database: Removed
- Network collectorhub_default: Removed
- Volume collectorhub_db_data: Removed

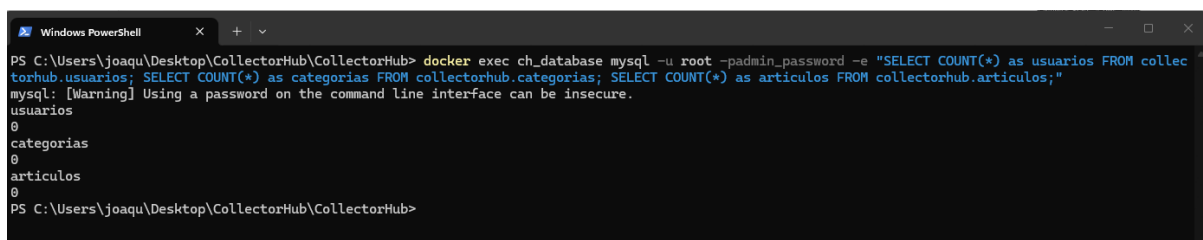


```
Windows PowerShell
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub> docker-compose down -v
time="2026-05-10T18:41:56+02:00" level=warning msg="C:\\Users\\joaqu\\Desktop\\CollectorHub\\CollectorHub\\docker-compose.yml: the attribute 'version'
is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] down 3/3
✓Container ch_database      Removed      1.6s
✓Network collectorhub_default Removed      0.3s
✓Volume collectorhub_db_data Removed      0.0s
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub> |
```

Volví a levantar el contenedor con docker-compose up -d y esperé a que MySQL terminara de arrancar. Al ejecutar los conteos el resultado fue:

usuarios=0, categorias=0, articulos=0.

La pérdida total de datos estaba confirmada.

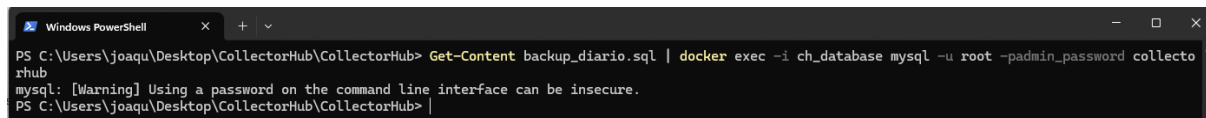


```
Windows PowerShell
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub> docker exec ch_database mysql -u root -padmin_password -e "SELECT COUNT(*) as usuarios FROM collec
torhub.usuarios; SELECT COUNT(*) as categorias FROM collectorhub.categorias; SELECT COUNT(*) as articulos FROM collectorhub.articulos;"
mysql: [Warning] Using a password on the command line interface can be insecure.
usuarios
0
categorias
0
articulos
0
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub>
```

9.5. Restauración desde el Backup

Con el contenedor corriendo pero la base de datos completamente vacía, ejecuté la restauración importando el archivo SQL que había generado antes. Como estaba en PowerShell usé Get-Content para redirigir el contenido del archivo al mysql del contenedor:

El comando terminó sin errores. Solo apareció el aviso habitual de MySQL sobre contraseñas en línea de comandos, que no afecta al resultado.



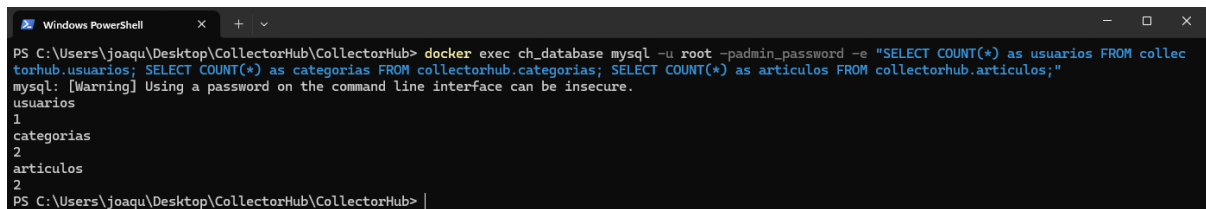
```
Windows PowerShell
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub> Get-Content backup_diario.sql | docker exec -i ch_database mysql -u root -padmin_password collectorhub
mysql: [Warning] Using a password on the command line interface can be insecure.
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub>
```

9.6. Verificación de Integridad

Por último comprobé que los datos habían vuelto exactamente igual que antes del borrado:

Resultado: usuarios=1, categorias=2, articulos=2.

Los números son idénticos a los del estado original antes de la destrucción. El simulacro confirmó que el plan funciona.



```
Windows PowerShell
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub> docker exec ch_database mysql -u root -padmin_password -e "SELECT COUNT(*) as usuarios FROM collectorhub.usuarios; SELECT COUNT(*) as categorias FROM collectorhub.categorias; SELECT COUNT(*) as articulos FROM collectorhub.articulos;"
mysql: [Warning] Using a password on the command line interface can be insecure.
usuarios
1
categorias
2
articulos
2
PS C:\Users\joaqu\Desktop\CollectorHub\CollectorHub>
```

10. Plan de Pruebas y Aseguramiento de Calidad

Para validar la estabilidad de Collector Hub, se ha combinado la evaluación unitaria en el lado del servidor con pruebas de integración manuales sobre el cliente.

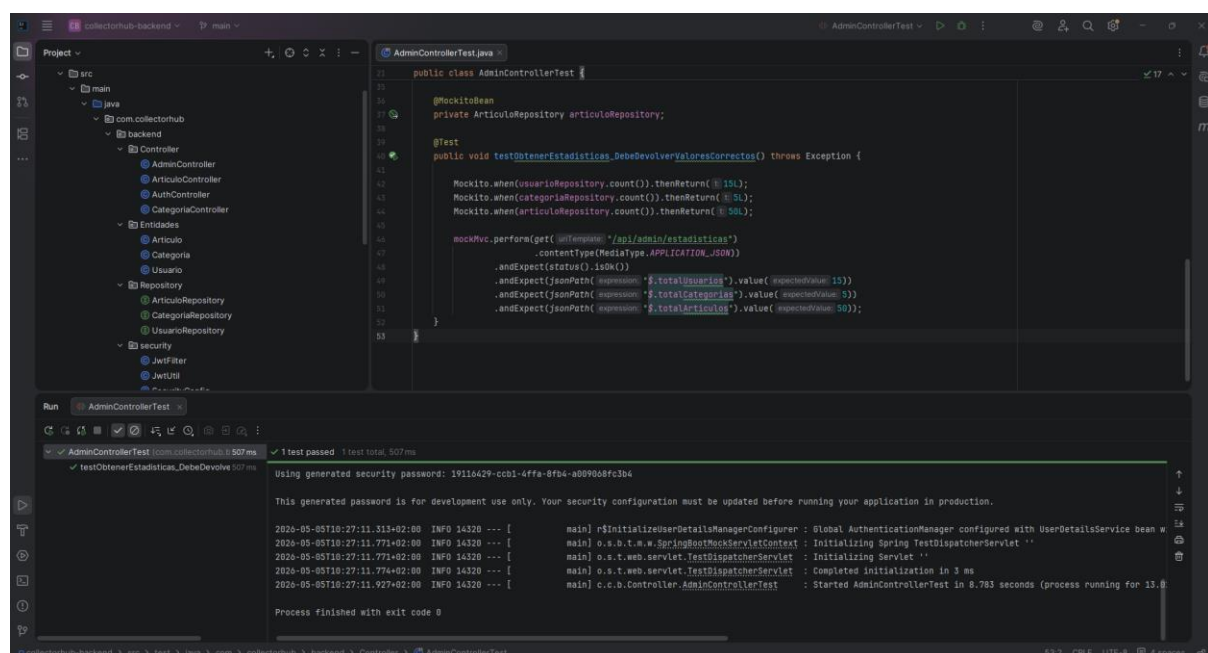
10.1. Pruebas Automáticas

Para validar la API REST sin depender del estado de la base de datos, se han implementado pruebas automáticas combinando **MockMvc** y **Mockito**. Con MockMvc simulamos las peticiones HTTP al servidor, mientras que con Mockito inyectamos datos falsos en los repositorios. Así comprobamos que la lógica del controlador funciona y devuelve los códigos de estado y el JSON correctos.

Caso 1: Panel de Administración (AdminControllerTest)

Se ha testado el endpoint de estadísticas globales. Para evitar que el filtro de seguridad bloqueara la prueba al no recibir un token real, se inyectó un mock de la clase JwtUtil. El test simula una petición GET a la ruta protegida y comprueba que el servidor devuelve un HTTP 200 (OK) con las cantidades exactas proporcionadas por los repositorios simulados.

Detalle de refactorización: Al trabajar con Spring Boot 3.4.x, la anotación habitual `@MockBean` generaba avisos de código obsoleto (deprecated). Para mantener el código limpio, se ha migrado a la nueva anotación estándar `@MockitoBean`.



The screenshot shows an IDE with the following components:

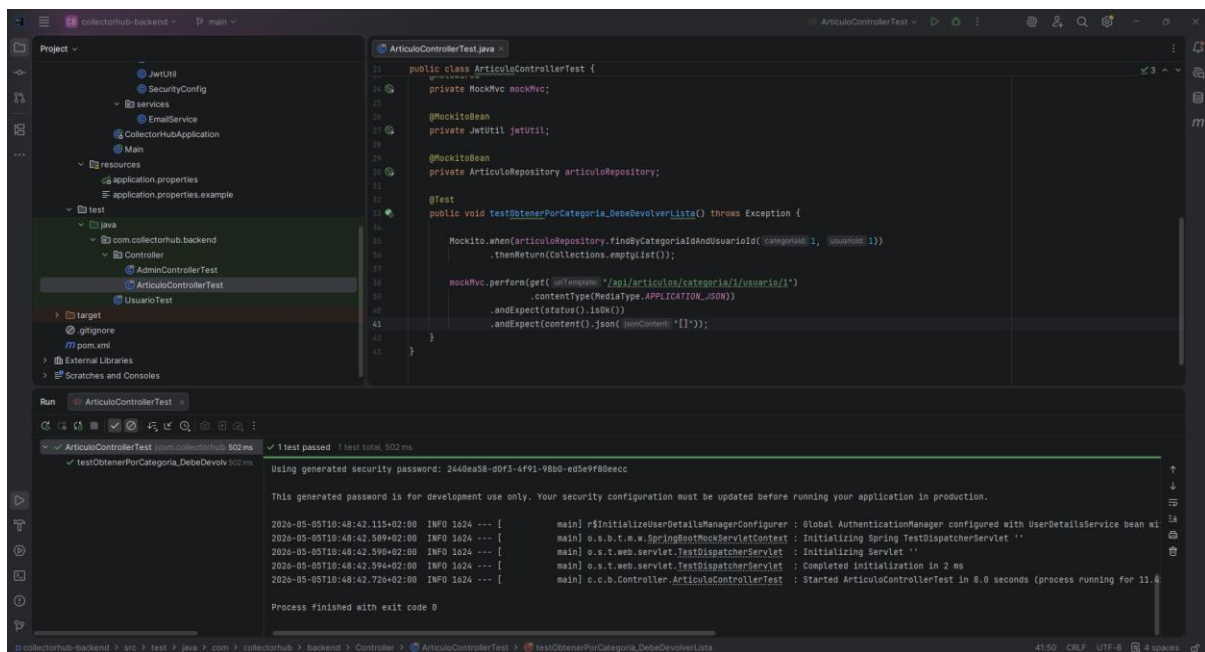
- Project Explorer:** Shows the project structure with packages like `com.collectorhub`, `Controller`, `Entidades`, `Repository`, and `security`.
- Editor:** Displays the `AdminControllerTest.java` file. The code includes:

```
21 public class AdminControllerTest {
22
23     @MockitoBean
24     private ArticleRepository articleRepository;
25
26     @Test
27     public void testObtenerEstadisticas_DebeDevolverValoresCorrectos() throws Exception {
28
29         Mockito.when(userRepository.count()).thenReturn(15L);
30         Mockito.when(categoriaRepository.count()).thenReturn(5L);
31         Mockito.when(articleRepository.count()).thenReturn(50L);
32
33         mockMvc.perform(get("/api/admin/estadisticas")
34             .contentType(MediaType.APPLICATION_JSON)
35             .andExpect(status().isOk())
36             .andExpect(jsonPath("$.totalUsuarios").value(expectedValue 15))
37             .andExpect(jsonPath("$.totalCategorias").value(expectedValue 5))
38             .andExpect(jsonPath("$.totalArticulos").value(expectedValue 50)));
39     }
40 }
```
- Run Console:** Shows the test execution results:

```
AdminControllerTest [com.collectorhub:0.0.0-SNAPSHOT]
testObtenerEstadisticas_DebeDevolverValoresCorrectos [OK]
Using generated security password: 19116429-cch1-4ffa-8f84-a0890a8fc3b4
This generated password is for development use only. Your security configuration must be updated before running your application in production.
2026-05-05T10:27:11.313+02:00 INFO 14320 --- [main] r$InitialUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsService bean w
2026-05-05T10:27:11.771+02:00 INFO 14320 --- [main] o.s.b.t.m.w.SpringBootMockServletContext : Initializing Spring TestDispatcherServlet ''
2026-05-05T10:27:11.772+02:00 INFO 14320 --- [main] o.s.t.web.servlet.TestDispatcherServlet : Initializing Servlet ''
2026-05-05T10:27:11.774+02:00 INFO 14320 --- [main] o.s.t.web.servlet.TestDispatcherServlet : Completed initialization in 3 ms
2026-05-05T10:27:11.927+02:00 INFO 14320 --- [main] c.e.b.Controller.AdminControllerTest : Started AdminControllerTest in 8.783 seconds (process running for 13.8)
Process finished with exit code 0
```

Caso 2: Controlador de Artículos (ArticuloControllerTest)

Se ha validado la lógica de filtrado de inventario a través del *endpoint* de lectura principal. Aislando la base de datos con `@MockitoBean` sobre `ArticuloRepository`, la prueba simula una petición GET a la ruta de búsqueda por ID de categoría y usuario. El test verifica que el controlador procesa los parámetros de la URL correctamente y responde con un código HTTP 200 (OK), devolviendo un array JSON válido sin generar excepciones de acceso a datos.



```
21 public class ArticleControllerTest {
22     private MockMvc mockMvc;
23
24     @MockitoBean
25     private JwtUtil jwtUtil;
26
27     @MockitoBean
28     private ArticleRepository articleRepository;
29
30     @Test
31     public void testObtenerPorCategoria_DebeDevolverLista() throws Exception {
32
33         Mockito.when(articleRepository.findByCategoryIdAndUserId(1, "usuario1"))
34             .thenReturn(Collections.emptyList());
35
36         mockMvc.perform(get("/api/articulos/categoria/1/usuario1")
37             .contentType(MediaType.APPLICATION_JSON))
38             .andExpect(status().isOk())
39             .andExpect(content().json("{\"content\": []}"));
40     }
41 }
42
43 }
```

Run ArticleControllerTest

✓ 1 test passed 1 test total, 502 ms

testObtenerPorCategoria_DebeDevolverLista 502 ms

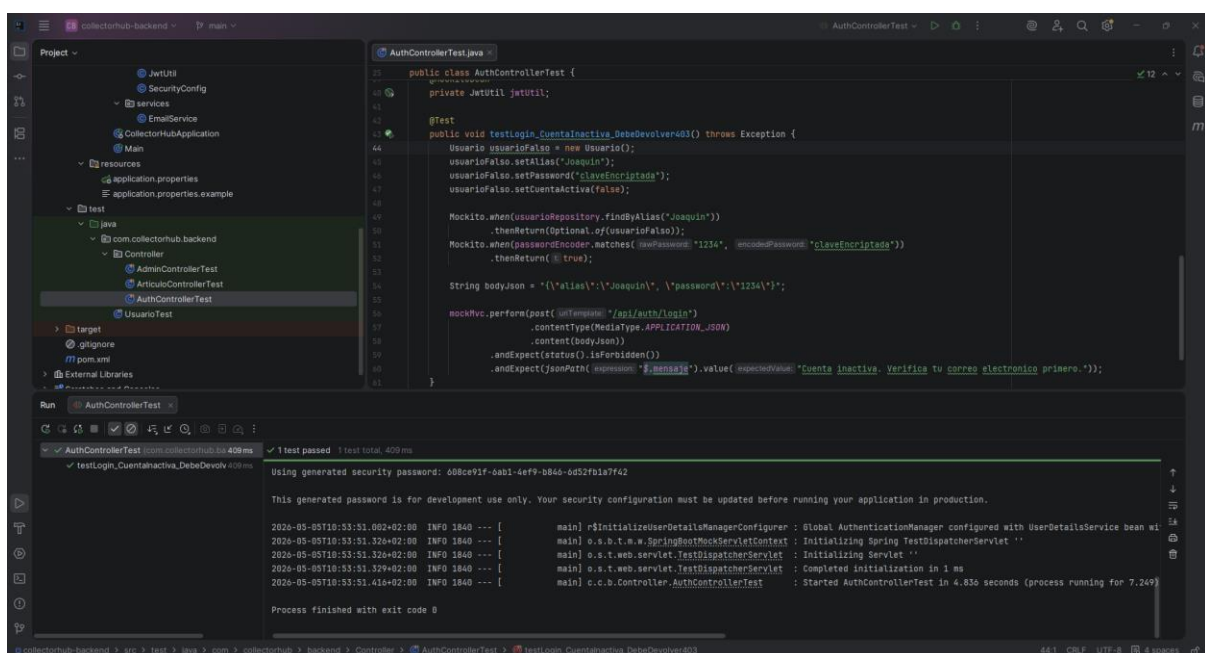
Using generated security password: 2640ea58-d0f3-4f91-9bb8-ed5e9f80eccc

This generated password is for development use only. Your security configuration must be updated before running your application in production.

```
2026-05-05T18:48:42.115+02:00 INFO 1624 --- [main] r$initializeUserDetailsManagerConfigurer : Global AuthenticationManager configured with UserDetailsServiceImpl bean wi
2026-05-05T18:48:42.589+02:00 INFO 1624 --- [main] o.s.b.t.m.w.SpringBootMockServletContext : Initializing Spring TestDispatcherServlet ''
2026-05-05T18:48:42.596+02:00 INFO 1624 --- [main] o.s.t.web.servlet.TestDispatcherServlet : Initializing Servlet ''
2026-05-05T18:48:42.594+02:00 INFO 1624 --- [main] o.s.t.web.servlet.TestDispatcherServlet : Completed initialization in 2 ms
2026-05-05T18:48:42.726+02:00 INFO 1624 --- [main] o.c.b.Controller.ArticleControllerTest : Started ArticleControllerTest in 8.0 seconds (process running for 11.4)
Process finished with exit code 0
```

Caso 3: Controlador de Autenticación (AuthControllerTest)

Se ha validado la lógica de seguridad y acceso mediante un caso de uso restrictivo. En lugar de probar un login exitoso estándar, la prueba simula un intento de inicio de sesión con credenciales correctas, pero con una cuenta que aún no ha verificado su código PIN. Se inyectaron *mocks* para aislar la base de datos, el encriptador (PasswordEncoder) y el servicio de mensajería (EmailService), evitando el envío de correos reales. El test confirma que el controlador bloquea la petición devolviendo el código HTTP 403 (Forbidden) y el aviso correspondiente, garantizando que no se emiten tokens JWT a cuentas inactivas.



The screenshot displays an IDE with the `AuthControllerTest.java` file open. The test class is as follows:

```
public class AuthControllerTest {  
    private JetUtil jetUtil;  
  
    @Test  
    public void testLogin_CuentaInactiva_DebeDevolver403() throws Exception {  
        Usuario usuarioFalso = new Usuario();  
        usuarioFalso.setAlias("Joaquin");  
        usuarioFalso.setPassword("claveEncriptada");  
        usuarioFalso.setCuentaActiva(false);  
  
        Mockito.when(usuarioRepository.findByAlias("Joaquin"))  
            .thenReturn(Optional.of(usuarioFalso));  
        Mockito.when(passwordEncoder.matches(usuarioFalso.getPassword(), usuarioFalso.getPassword()))  
            .thenReturn(true);  
  
        String bodyJson = "{ \"alias\": \"Joaquin\", \"password\": \"claveEncriptada\" }";  
  
        mockMvc.perform(post("/api/auth/login")  
            .contentType(MediaType.APPLICATION_JSON)  
            .content(bodyJson))  
            .andExpect(status().isForbidden())  
            .andExpect(jsonPath("$.mensaje").value("Cuenta inactiva. Verifica tu correo electrónico primero."));  
    }  
}
```

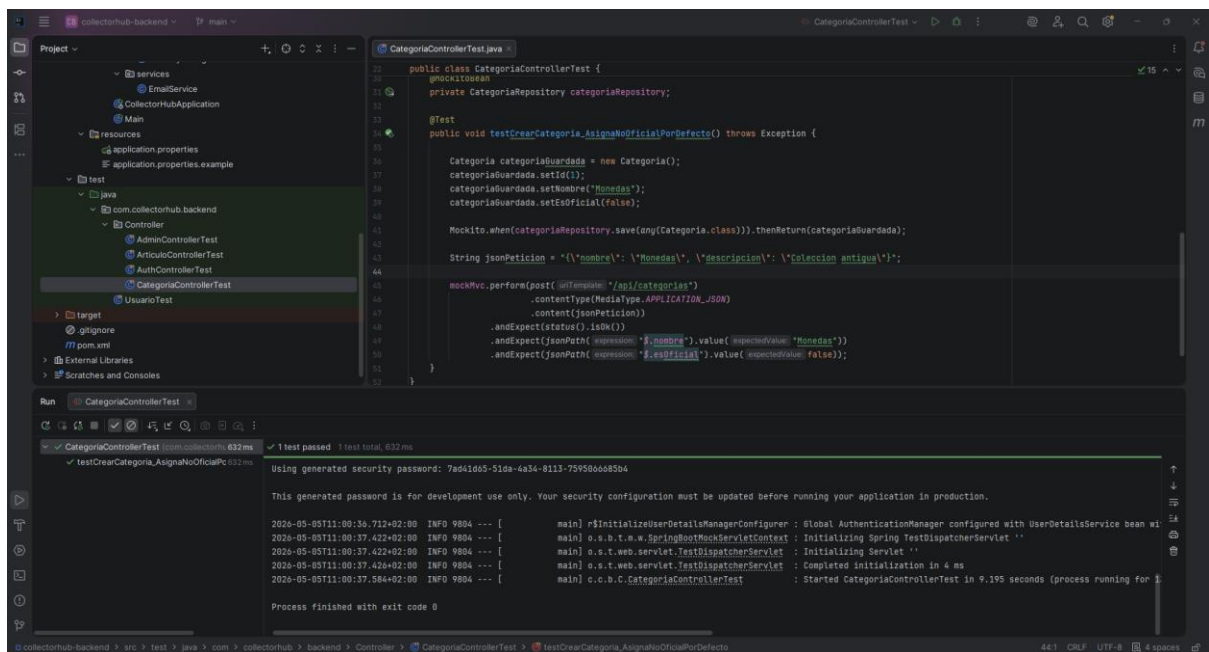
The Run window shows the test results:

```
AuthControllerTest [com.collectorhub.ba] 409 ms  
✓ testLogin_CuentaInactiva_DebeDevolver403 409 ms  
Using generated security password: 608ce91f-6ab1-4ef9-b86d-0d52f61b7f42  
This generated password is for development use only. Your security configuration must be updated before running your application in production.  
2026-05-05T10:53:51.002+02:00 INFO 1840 --- [main] r$initializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsServiceImpl bean  
2026-05-05T10:53:51.326+02:00 INFO 1840 --- [main] o.s.b.t.w.s.SpringBootMockServletContext : Initializing Spring TestDispatcherServlet ''  
2026-05-05T10:53:51.328+02:00 INFO 1840 --- [main] o.s.t.w.s.TestDispatcherServlet : Initializing Servlet ''  
2026-05-05T10:53:51.379+02:00 INFO 1840 --- [main] o.s.t.w.s.TestDispatcherServlet : Completed initialization in 1 ms  
2026-05-05T10:53:51.416+02:00 INFO 1840 --- [main] c.c.b.Controller.AuthControllerTest : Started AuthControllerTest in 4.836 seconds (process running for 7.249)  
Process finished with exit code 0
```

Caso 4: Controlador de Categorías (CategoriaControllerTest)

La validación de este controlador se enfocó en el *endpoint* de inserción (POST) para confirmar el cumplimiento de las reglas de negocio en la capa de servicio.

Específicamente, se testeó la lógica que asigna por defecto el valor booleano `false` al atributo `esOficial` cuando este es omitido en el cuerpo de la petición. Mediante la simulación del método `.save()` en el `CategoriaRepository`, el test confirma que la API procesa el JSON incompleto, aplica la regla por defecto y devuelve un código HTTP 200 (OK) con la entidad correctamente formateada.



The screenshot displays an IDE with the following components:

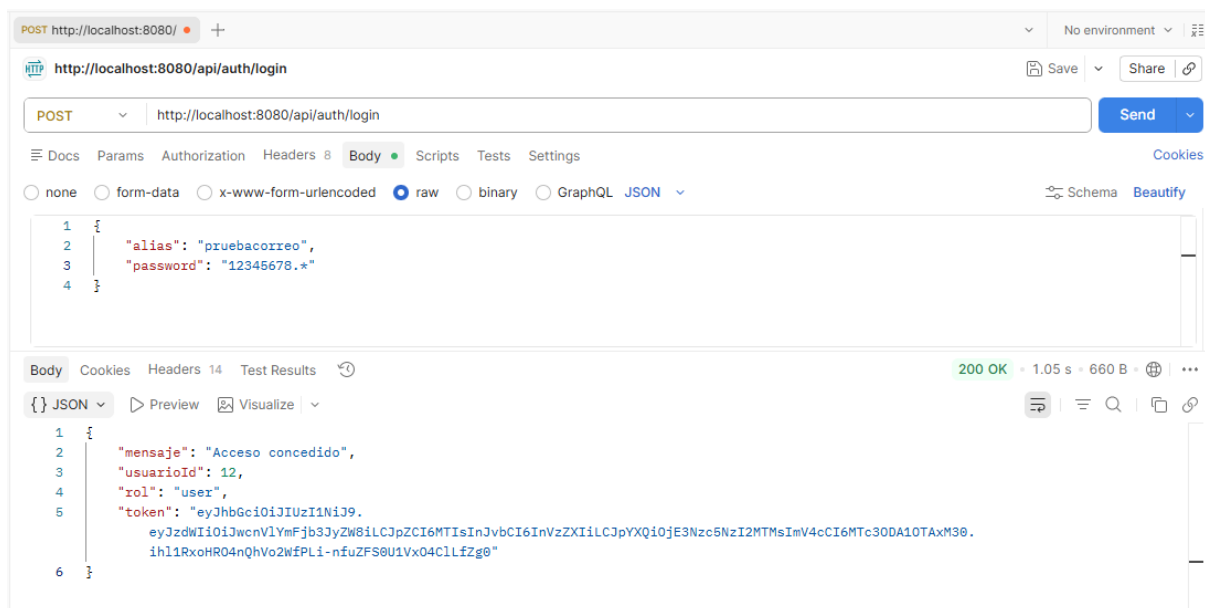
- Project Explorer:** Shows the project structure with folders for `services`, `resources`, `test`, and `target`. The `test` folder contains `com.collectorhub.backend` and `Controller`, with `CategoriaControllerTest` selected.
- Editor:** Displays the `CategoriaControllerTest.java` file. The code includes a `@MockitoSetup` annotation, a `@Test` method `testCrearCategoria_AsignaNoOficialPorDefecto()`, and a `mockMvc` call to perform a POST request to `/api/categorias` with a JSON body. The test expects a status of 200 and specific values for `nombre` and `esOficial`.
- Run Console:** Shows the execution of the test. It indicates that the test passed and displays a generated security password. Log messages show the initialization of the Spring TestDispatcherServlet and the completion of the test.

10.2. Pruebas Manuales de Integración

Para complementar la evaluación automatizada y aportar evidencias reales de ejecución sobre el entorno levantado, se ha utilizado el cliente HTTP Postman. El objetivo es certificar que la API responde adecuadamente a peticiones externas y gestiona correctamente los códigos de estado HTTP frente a escenarios de éxito y de error forzado.

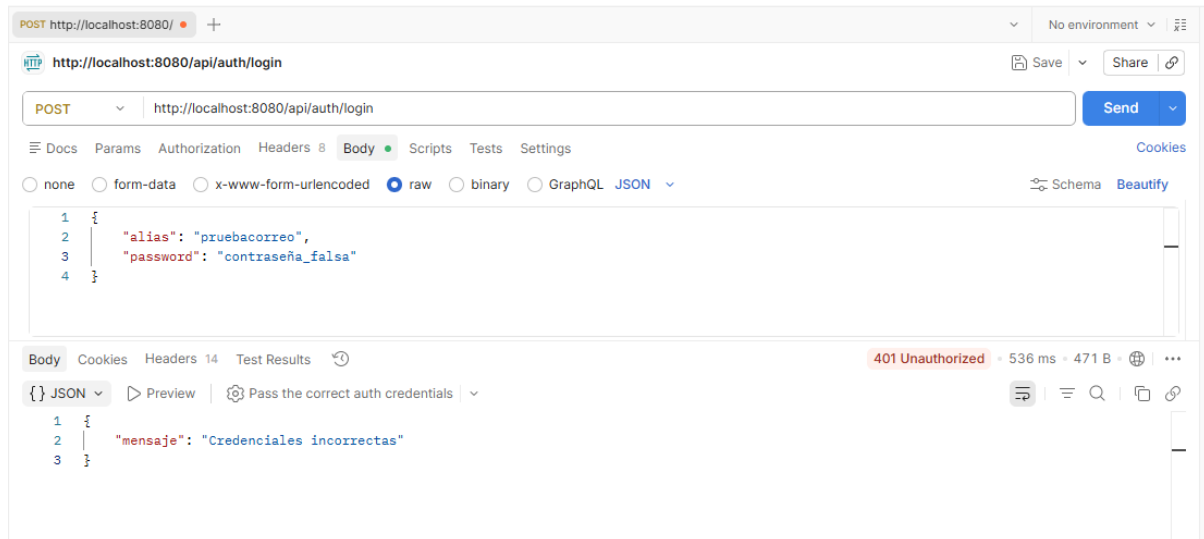
1: Autenticación exitosa y emisión de JWT

Se realizó una petición POST al *endpoint* `/api/auth/login` enviando credenciales válidas en el cuerpo de la petición. El servidor procesó la solicitud, validó el estado de la cuenta (PIN verificado) y devolvió un código **200 OK** junto con el token JWT necesario para acceder a las rutas protegidas.



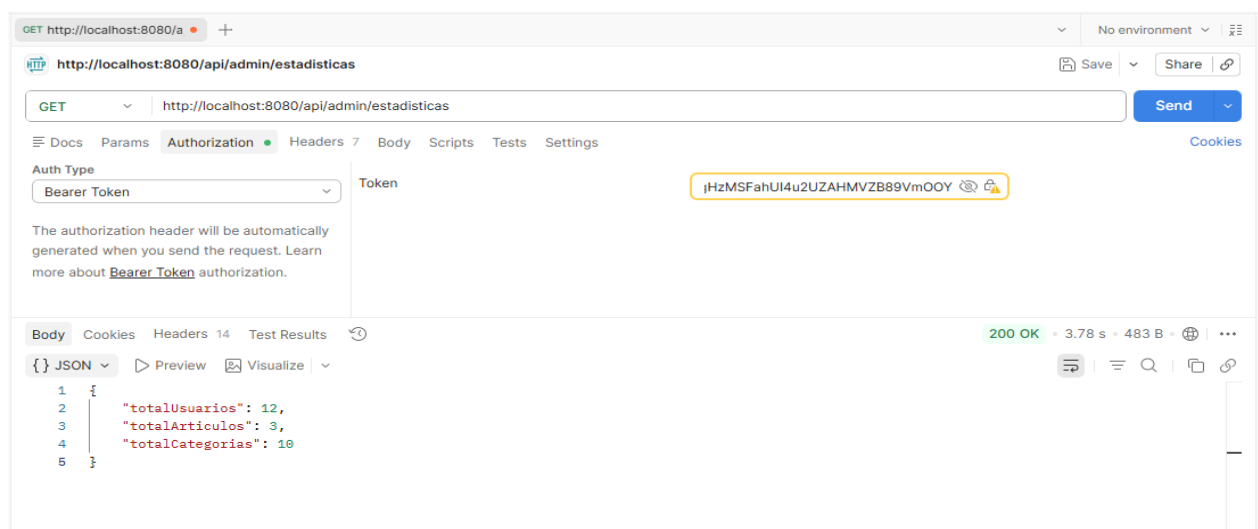
2: Gestión de acceso no autorizado

Se comprobó la robustez del sistema de seguridad forzando una petición POST al mismo *endpoint* con credenciales inválidas. El controlador interceptó correctamente el fallo de validación de *BCrypt* y denegó el acceso, devolviendo un código **401** **Unauthorized** sin exponer información sensible del sistema.



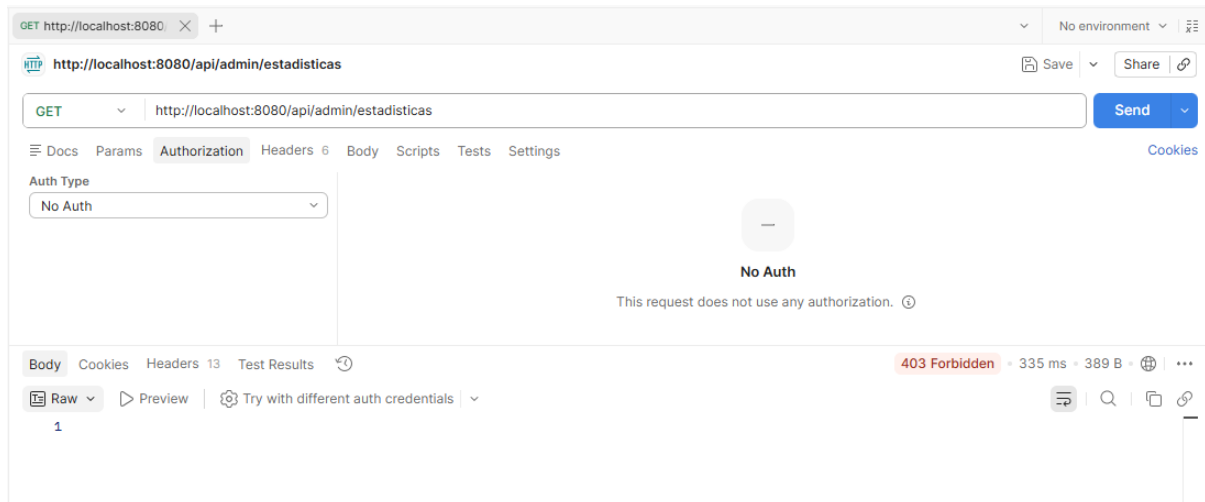
3: Acceso autorizado mediante Bearer Token

Se validó el correcto funcionamiento del filtro JWT (*Zero Trust*). Se realizó una petición GET al panel de administración (`/api/admin/estadisticas`), inyectando el token JWT en la cabecera `Authorization: Bearer <token>`. El servidor descifró el *claim* de seguridad, validó la firma y devolvió un código **200 OK** con la estructura JSON de métricas correspondiente.



4: Bloqueo perimetral de rutas protegidas

Para certificar la inviolabilidad de los *endpoints*, se simuló un ataque de acceso directo a la misma ruta administrativa (`/api/admin/estadisticas`) deshabilitando la cabecera de autorización. El sistema respondió de forma esperada con un estado **403 Forbidden / 401 Unauthorized**, abortando la petición antes de llegar al controlador y protegiendo el acceso a la capa de datos.



5: Persistencia de datos mediante POST

Finalmente, se probó la capacidad de inserción del sistema ejecutando una petición POST contra `/api/categorias`, incluyendo un *payload* JSON válido y el token de sesión. El servidor procesó la deserialización del objeto, persistió la entidad a través de Hibernate en MySQL, y resolvió la petición con un **200 OK**, devolviendo la entidad generada con su nueva clave primaria asignada.

